

Friedrich-Alexander-Universität
Erlangen-Nürnberg (FAU)

NHR@FAU

Assignment 3: Data transfer

Programming Techniques for Supercomputers (PTfS)

Muhammad Omar
Student IdM: to37gulo
Student HPC-Account: ptfs356v

Gehan Younes
Student IdM: qi39dyry
Student HPC-Account: ptfs437v

May 20, 2026

Q1: Cache Optimization (10 crd)

Verdict: The advice is wrong.

Cache blocking benefits kernels with *temporal* reuse — the same element accessed multiple times. In this matrix-sum kernel, every element of `m` is read *exactly once*; there is nothing to keep in cache between accesses.

Why the original code is optimal. In C, `m[r][c]` is row-major, so the inner loop over `c` is unit-stride. Every 64-byte cache line delivers 8 useful doubles — perfect spatial locality. The hardware prefetcher recognizes the long contiguous stream and hides memory latency.

Why tiling is at best equal, more often worse.

- **Same data volume.** Every element is still transferred once; bytes-per-Flop is identical.
- **Shorter streams.** With tile size b, the inner c-loop runs only b iterations before jumping $N_{\text{cols}} \times 8$ bytes. Small b defeats the hardware prefetcher, making the loop latency-limited.
- **Extra overhead.** Four loops instead of two, with non-trivial index arithmetic.

When does blocking help? Only for kernels with genuine temporal reuse, e.g. dense matrix–matrix multiplication ($O(N^3)$ work, $O(N^2)$ data $\Rightarrow$ $O(N)$ reuse per element) or multi-sweep stencils. A single-pass summation has *no* reuse, so blocking cannot help.

Q2: Intricate Stencil Code on Ivy Bridge (30 crd)

Operation count per scalar inner iteration

Sub-expression	MUL	ADD
$2 \cdot V$ (scale)	1	0
$2V - U$	0	1
$c_0V,\ c_1S_1,\ \ldots,\ c_4S_4$ (5 scalings)	5	0
$S_d = \sum_6 V$ (4 depth-d sums, 5 adds each)	0	20
Sum $c_0V + \ldots + c_4S_4$ (4 adds)	0	4
$C \cdot (\ldots)$	1	0
Final accumulation into U	0	1
Total	**7**	**26**

Total: $F = 33$ Flop/scalar iteration.

Loads/stores per scalar iteration. V accesses: 1 (center) $+8 \times i + 8 \times j + 8 \times k = 25$. Plus 1 load of U, 1 load of C, 1 store of U $\Rightarrow$ **27 loads + 1 store**.

Ivy Bridge architectural assumptions

- 1 AVX MUL/cy (port 0), 1 AVX ADD/cy (port 1) — independent, fully pipelined.
- L1 load: 2 × 16 B ports = 1 AVX (256-bit) load per cycle.
- AVX width: $W = 4$ doubles.

Bottleneck analysis

AVX (vector) execution. One AVX iteration covers $W = 4$ scalar iterations.

Resource	Ops	Cycles
L1 load ports (1 AVX load/cy)	27 vector loads	**27**
Store port	1	1
ADD port	26	26
MUL port	7	7

Bottleneck: **L1 load throughput** (27 cy per AVX iteration, delivering $4 \times 33 = 132$ Flop).

$$\boxed{P^{\max}_{\text{AVX}} = \frac{132 \text{ Flop}}{27 \text{ cy}} \approx 4.89 \text{ Flop/cy}}$$

This is $\approx 61\%$ of AVX peak (8 Flop/cy).

Scalar execution. Two scalar loads per cycle $\Rightarrow$ 27 loads take only ≈ 14 cy. The bottleneck shifts to the **ADD port** (26 adds at 1 ADD/cy).

$$\boxed{P^{\max}_{\text{scalar}} = \frac{33 \text{ Flop}}{26 \text{ cy}} \approx 1.27 \text{ Flop/cy}}$$

Summary.

Mode	Cycles/inner iter	Flop/cy	Bottleneck
Scalar	26	≈ 1.27	ADD port (port 1)
AVX	27	≈ 4.89	L1 load throughput

Vectorization gives $\approx 3.9\times$ speedup (not the ideal $4\times$ because vectorizing the 26 ADDs shifts the bottleneck to the load ports).

Q3: Dense MVM — Locality Analysis (15 crd)

`for(i) for(j) c[i] += a[i][j]*b[j];` $N = 15000$, doubles

Sizes: $|a| \approx 1.68$ GiB, $|b| = |c| \approx 117$ KiB.

Array	Spatial locality	Temporal locality
$a[i][j]$	Excellent — unit-stride inner loop (row-major)	None — each element read once; $\|a\| \gg$LLC
$b[j]$	Excellent — unit-stride inner loop	High — reused N times across outer loop; $\|b\| = 117$ KiB fits L2
$c[i]$	Good — consecutive lines across outer iterations	Perfect — held in register for full inner loop

Key observations. $b[j]$ exhibits N-fold temporal reuse: after the first outer iteration b fits in L2 (256 KiB) and stays cached. $c[i]$ is kept in a register accumulator by the compiler — only one load and one store per outer iteration. The only unmitigated DRAM traffic is the matrix a.

Code balance and roofline. With only a coming from memory:

$$B_c = \frac{8\text{ B (load of } a)}{2\text{ Flop}} = 4\text{ B/Flop} \quad \Rightarrow \quad P = \frac{b_S}{B_c} = \frac{16\text{ GB/s}}{4\text{ B/F}} = 4\text{ GFlop/s}$$

which equals single-core peak — the kernel is memory-bound but can in principle reach peak, precisely because b stays cached.

Q4: Loop Nests

(a) What is wrong and how to fix it (10 crd)

```
for (int i=0; i<N; i++)
  for(int j=0; j<N; j++)
    s += a[j][i];  // inner index is FIRST subscript -- stride-N access!
```

The inner loop variable `j` indexes the *first* (row) subscript. In C (row-major storage), incrementing `j` by 1 advances the address by $N \times 8\text{ B} = 16384 \times 8 = 128$ KiB. Consequences:

- Every inner iteration is a cache miss; only 1 of 8 doubles per cache line is used.
- The hardware prefetcher cannot recognize the strided pattern.
- Up to $N = 16384$ distinct 4 KiB pages touched per inner loop ⇒ TLB thrashing.

Fix — swap the loop indices so the unit-stride direction is innermost:

```
s = 0.0;
for (int j=0; j<N; j++)
  for (int i=0; i<N; i++)
    s += a[j][i];   // now unit-stride: inner index is rightmost subscript
```

(Equivalently, keep the loop order but write `a[i][j]`.) The swap is safe because summation is commutative.

(b) Maximum bandwidth without prefetching (15 crd)

Without prefetching, at most *one* cache-line miss is outstanding at a time. The core issues a miss, waits for the full memory latency T_l, then issues the next. Achievable bandwidth:

$$b_{\text{no-PF}} = \frac{L_{\text{line}}}{T_l}$$

Default $L_{\text{line}} = 64$ B:

$$b_{\text{no-PF}} = \frac{64\text{ B}}{100\text{ ns}} = 0.64\,\text{GB}\,\text{s}^{-1} \quad (0.5\%\text{ of }128\text{ GB/s})$$

Doubled $L_{\text{line}} = 128$ B:

$$b_{\text{no-PF}}^{(128)} = \frac{128\text{ B}}{100\text{ ns}} = 1.28\,\text{GB}\,\text{s}^{-1}$$

Doubling the line size doubles the achievable bandwidth linearly, because the same single in-flight transfer now delivers twice as much useful data per latency window. The bus is still essentially idle (1% of peak).

(c) Outstanding misses required to saturate 128 GB/s (20 crd)

Saturating memory bandwidth requires memory-level parallelism. By **Little's law** (see lecture: $\#\mathrm{PF} = T/(V/b)$):

$$N_{\mathrm{miss}} = \frac{b_S \cdot T_l}{L_{\mathrm{line}}} = \frac{128 \times 10^9 \text{ B/s} \times 100 \times 10^{-9} \text{ s}}{64 \text{ B}} = \frac{12800}{64} = \boxed{200 \text{ outstanding cache-line misses}}$$

Additional prerequisites.

1. **Hardware MLP capacity.** A single core has only $\sim$ 10–12 line-fill buffers (LFBs). Reaching 200 concurrent misses requires the *hardware streamer/prefetcher* (which uses additional L2/L3 buffers) and/or explicit **software prefetch** instructions far ahead of the demand stream. In practice, the full 128 GB/s usually requires multiple cores.

2. **Unit-stride access pattern.** Only after the loop-order fix (part a) does the L1/L2 hardware streamer recognize a long contiguous stream and issue many prefetches. The original strided pattern disables the prefetcher entirely.

3. **Sufficient out-of-order window.** The out-of-order engine must find $\sim$ 200 independent load misses to fill that many LFB entries. This requires a large enough reorder buffer and load queue, and the absence of long dependency chains — satisfied here since the sum reduction compiles to multiple independent partial accumulators.

4. **No write-allocate overhead / aligned accesses.** Each miss must cost exactly one line transfer.

5. **Sufficient TLB coverage.** Large pages (2 MiB) prevent page-walk serialization of the miss stream.

6. **Enough open DRAM banks.** The memory controller must be able to service 200 in-flight requests in parallel; otherwise effective latency increases and more concurrency is needed.